Where Does Model Folding Break? A Reproduction and Stress Test of Data-Free Variance Repair on ResNet18/CIFAR10

Donggeon Kim
20242752
POSTECH
dgkim1403@postech.ac.kr

Abstract

This report presents a stress test of the ICLR 2025 paper "Forget the Data and Fine-tuning! Just Fold the Network to Compress". Model folding is a compression method that does not require training data: it clusters similar channels, merges them, and repairs the resulting variance collapse. The data-free repair variant, Fold-AR, depends on a strong assumption that the input channels of each layer are uncorrelated. In this Track-2 study, I (i) reproduce the three core variants (Fold-Naive, Fold-AR, Fold-R) on ResNet18/CIFAR10 using the authors' public code, (ii) document four compatibility patches that were needed to run the released code on a modern Python 3.12 / PyTorch 2.x environment, and (iii) empirically locate the sparsity range where Fold-AR breaks away from the data-driven upper bound Fold-R. I find that Fold-AR closely follows Fold-R up to about 35% layer-wise sparsity, but breaks down sharply between 45% and 75%. This is consistent with the uncorrelated-input assumption failing as the clusters become larger. I also attempted to extend the analysis to VGG11-BN, but the VGG sweep did not complete due to an unresolved runtime issue. Code and logs: `https://github.com/posdgkim-collab/model-folding-stress-test`.

1 Introduction

Deep neural networks keep getting larger as their accuracy improves. This makes them hard to use in environments with limited hardware resources, so model compression methods are used to reduce the model size while keeping the accuracy as close to the original as possible. Common compression methods include the following. **Pruning** removes weights, neurons, or channels that are considered unimportant. **Quantization** represents weights or activations with fewer bits to reduce memory cost. **Fine-tuning** retrains the compressed model on training data to recover the accuracy that was lost during compression.

This paper proposes **model folding** [1], which also compresses the network by merging similar neurons or channels. The procedure has three main steps: cluster similar channels, merge each cluster into one, and then repair the variance collapse or overshooting that the merging step causes.

The typical way to do the repair step is a data-driven approach. The model forwards real training data, the activation mean and variance before and after folding are measured directly, and the internal statistics of the folded model are matched to the original. This means that the compression method itself depends on having access to the training data. In many realistic situations the training data is not available anymore: medical data has privacy issues, financial or internal company data has confidentiality issues, and so on. In these situations only the trained model checkpoint is available, so data-driven compression methods have a limitation.

Unlike the standard data-driven model folding, this paper performs folding without training data. One of the biggest problems during compression is that the internal data statistics get broken: when the mean and variance of internal activations are changed, the accuracy drops. The key requirement of model folding is that the variance ratio between the compressed model and the uncompressed model maps close to one. When data is available, the internal statistics of the folded model can be tuned against the original data, but in the data-free setting the intra-cluster mean correlation must be estimated from the weights alone. This estimation relies on an assumption that the input channels are uncorrelated.

This report performs a stress test of this paper using the authors' public code, following Track 2 of the project guideline. The two main analyses are:

- How does the accuracy of the data-free repair (Fold-AR) compare to the data-driven repair (Fold-R) across the full sparsity range?
- At what sparsity does the uncorrelated-input assumption start to break down sharply?

I also attempted to extend the analysis to a second architecture (VGG11-BN) to check whether the breakdown observed on ResNet18 generalizes, but the VGG sweep did not complete in practice (Section 5).

2 Background

Model folding. Fold-AR first absorbs the BatchNorm normalization into the current layer weight to obtain a normalized weight $\hat{W}_l = \Sigma_n W_l$. Because folding the current layer also affects the next layer, the procedure considers both layers together. The combined matrix is

$$W_{\text{tot}} = [\, W_{l+1}^\top \mid \hat{W}_l \mid \operatorname{diag}(\Sigma_s) \,].$$

Similar channels are then grouped by k-means clustering:

$$U = \arg\min \|W_{\text{tot}} - U(U^\top U)^{-1} U^\top W_{\text{tot}}\|_F^2.$$

Once the cluster assignment U is fixed, the weights are reduced as

$$\begin{aligned} \Sigma_s &\leftarrow (U^\top U)^{-1} U^\top \Sigma_s U \\ W_{l+1}^\top &\leftarrow U^\top W_{l+1}^\top \\ \hat{W}_l &\leftarrow (U^\top U)^{-1} U^\top \hat{W}_l. \end{aligned}$$

That is, multiple output channels of the current layer are merged into the cluster centroid, the next layer's input channels are merged accordingly to match the reduced channel count, and the BatchNorm scale is also merged at the cluster level. This shrinks the network structurally.

Variance repair. When folding averages the channels in a cluster, the variance of the centroid can become smaller than before folding. This causes variance collapse or overshooting, which lowers accuracy, so a corrective scaling factor is needed:

$$\sqrt{\frac{N_c}{N_c + (N_c^2 - N_c)E[c]}}.$$

When $E[c] = 0$ the channels are completely uncorrelated, and the factor becomes $\sqrt{N_c}$. When $E[c] = 1$ the channels are completely correlated, and the factor becomes 1.

Why Fold-AR needs an assumption. Fold-R can compute the activation correlation directly because real data is available. Fold-AR cannot, because no data is available. The paper therefore assumes that the layer-output channels are mutually uncorrelated. Under this assumption, the input covariance matrix needed by the formula can be simplified to (essentially) the identity, so the activation correlation does not need to be measured.

Concretely, suppose channels i and j are being folded. Their pre-activations are

$$\tilde{x}_i = w_i x, \qquad \tilde{x}_j = w_j x,$$

where x is the output activation of the previous layer, and w_i, w_j are the normalized weight vectors of channels i and j. To compute $\mathrm{Corr}(\tilde{x}_i, \tilde{x}_j)$ exactly, one needs to know how x is distributed, i.e. the actual data. With data, the correlation is

$$E[c] \;\approx\; \text{average over } i,j \text{ in cluster of} \quad \frac{w_i \Sigma_x w_j^\top}{\sqrt{(w_i \Sigma_x w_i^\top)(w_j \Sigma_x w_j^\top)}},$$

where Σ_x is the covariance matrix of the previous layer's activation. This describes how the channels of the previous layer move together, and computing it requires forwarding real images.

If we instead assume that the previous layer's activation channels are uncorrelated, then $\Sigma_x \approx I$ (the off-diagonal terms of the covariance matrix are zero). The correlation simplifies to

$$\mathrm{Corr}(\tilde{x}_i, \tilde{x}_j) \;\approx\; \frac{w_i \cdot w_j}{\|w_i\| \|w_j\|}.$$

This depends only on the weights, so it can be evaluated without data.

REPAIR variants studied. I evaluate three folding variants in this report.

- **Fold-Naive:** clustering and merging are performed, but no repair is applied.
- **Fold-AR:** data-free repair. $E[c]$ is estimated from the weights, under the uncorrelated-input assumption. Centroids are rescaled accordingly.
- **Fold-R:** data-driven repair. Training data is forwarded through the compressed network to rebuild the internal statistics.

3 Code environment and patches

There are two GitHub repositories associated with the paper:

- "Universal" version (includes LLM support): `https://github.com/nanguoyu/model-folding-universal`
- "modelfolding_cv" version: `https://github.com/marza96/ModelFolding` (commit `33e2818`, which the universal README recommends for CV reproduction).

The universal version's README recommends the latter for reproducing the CV results, so I used the latter.

All experiments were run on a Google Colab T4 GPU with PyTorch 2.x and Python 3.12.

The release was written against Python 3.8 and an older PyTorch version, so it does not run on the current Colab environment as-is. Four compatibility issues had to be patched.

(1) `hkmeans` build failure. The Hartigan k-means package required by the paper does not build on Python 3.12 because its `versioneer.py` calls `configparser.SafeConfigParser`, which was removed in Python 3.12. I replaced `HKMeans` with `sklearn.cluster.KMeans`. Most of the arguments map one-to-one, and the only argument I had to drop was `n_jobs=-1`, which sklearn's KMeans does not accept. Since the paper only describes the method as "k-means clustering" and gives no specific advantage to the Hartigan variant, I expect this replacement not to change the results substantially.

(2) Removed exception class. `utils/utils.py` catches `torch.nn.modules.module.ModuleAttributeError`, but this class was removed in PyTorch 2.x. The original intent was to fall through for ResNet18 BasicBlocks, which have no `bn3`. I replaced it with the base class `AttributeError`.

(3) Missing dependency. `thop` is imported by the main experiment script but is not listed in `requirements.txt`, so I installed it separately.

(4) Checkpoint incompatibility. The universal repo's README recommends the `PyTorch_CIFAR10` [1] pretrained ResNet18 (reported accuracy 93.07%), but this model has a different architecture from the ResNet18 in the marza96 repo. In particular, the `PyTorch_CIFAR10` model has an initial `maxpool` and a single ReLU per BasicBlock, while the marza96 ResNet18 has no initial maxpool and two ReLUs per BasicBlock. All 122 state-dict keys still match under the `{downsample:shortcut, fc:linear}` mapping that the authors provide, but the forward graph is different. Loading the public checkpoint into the marza96 ResNet18 gives only $\sim 21\%$ accuracy even before any compression.

I therefore retrained ResNet18 on CIFAR10 from scratch with the following recipe: SGD with momentum 0.9, cosine schedule starting at learning rate 0.1, weight decay 5×10^{-4}, batch size 128, 60 epochs. The final test accuracy is **94.57%**, and this model is used as the baseline checkpoint in all experiments below.

4 Experiments

4.1 Setup

The model is the marza96 ResNet18 (11,173,962 parameters) trained from scratch with the recipe above. Final test accuracy: 94.57%. The same checkpoint is used for all three folding variants.

I sweep layer-wise sparsity over 14 points:

$$\{0.01,\ 0.025,\ 0.05,\ 0.075,\ 0.10,\ 0.15,\ 0.25,\ 0.35,\ 0.45,\ 0.55,\ 0.65,\ 0.75,\ 0.85,\ 0.95\},$$

applied uniformly across layers, matching the sweep used by the release script.

4.2 Reproducing the three REPAIR variants

The accuracy ordering reported by the paper holds in my reproduction:

$$\text{Fold-R} \ \succeq\ \text{Fold-AR} \ \succeq\ \text{Fold-Naive}.$$

All three variants are essentially equivalent up to about 10% sparsity, and the gaps start to widen rapidly after about 25%. Some specific numbers:

- At 25% sparsity: Fold-R 92.16%, Fold-AR 90.97%, Fold-Naive 84.42%.
- At 45% sparsity: Fold-R 84.34%, Fold-AR 70.65%, Fold-Naive 28.47% (random chance is 10%).
- At 75% sparsity: Fold-R 28.84%, Fold-AR 12.14%, Fold-Naive 10.76%.

The absolute accuracies in my setting are higher than those reported in the paper, because my baseline model (94.57%) is itself stronger than the published checkpoint (93.07%). The qualitative trends, however, match Figure 5 of the original paper.

4.3 Where does Fold-AR break?

The headline question of this stress test is up to what sparsity Fold-AR can keep up with Fold-R, and at what sparsity the uncorrelated-input assumption breaks. To make this concrete, I define the gap

$$\Delta(s) = \text{Fold-R}(s) - \text{Fold-AR}(s),$$

shown in Figure 2.

Three points stand out.

- Up to about 35% sparsity, the difference between Fold-R and Fold-AR is small. In this regime the data-free repair can substitute for the data-driven repair without much loss.
- Around 55% sparsity, a clear breakdown appears. The gap between Fold-R and Fold-AR grows to about 32 percentage points in this region.
- Above 85% sparsity, both variants collapse to near random chance. The limitation here is no longer about REPAIR but about the folding step itself.

[1] `https://github.com/huyvnphan/PyTorch_CIFAR10`

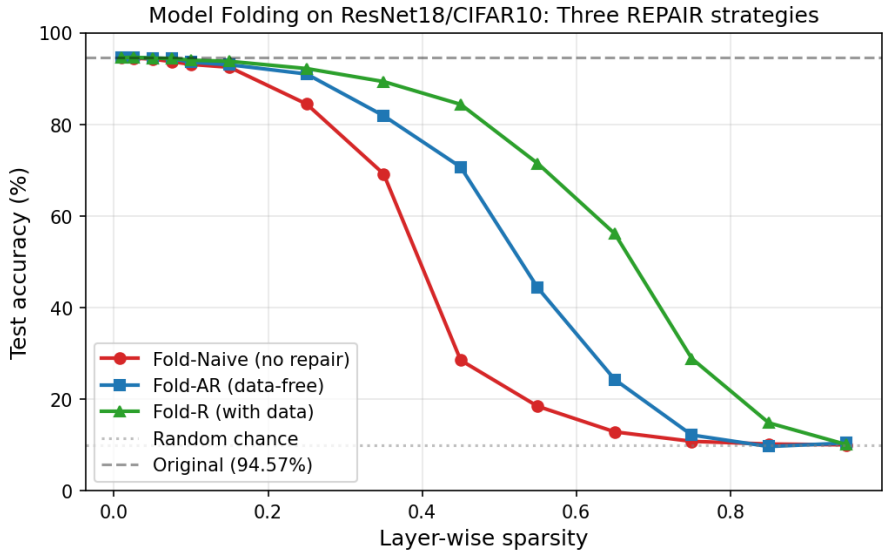

Figure 1: Reproduction of the three REPAIR variants on ResNet18/CIFAR10. All three variants are statistically equivalent up to $\sim 10\%$ sparsity. Fold-AR closely tracks the data-driven Fold-R up to $\sim 35\%$ sparsity and then diverges sharply. Random chance (10%) and the uncompressed baseline (94.57%) are marked.

4.4 Why does the breakdown happen around 55%?

As shown in Section 2, Fold-AR scales each centroid by

$$\hat{z}_l(c) \;\leftarrow\; \hat{z}_l(c) \cdot \sqrt{\frac{N_c}{N_c + (N_c^2 - N_c)E[c]}}.$$

When the input channels are truly uncorrelated, this scaling exactly cancels the variance collapse caused by averaging. But as sparsity grows, two effects make the uncorrelated-input assumption worse, and I think they jointly explain the $\sim 55\%$ breakdown.

The first reason is that the cluster size grows. At low sparsity, each cluster contains only a small number of channels, and pairing near-duplicate channels is reasonable. As sparsity increases, more and more channels are forced into the same cluster, and channels that are not really similar end up grouped together. This means the intra-cluster correlation $E[c]$ that Fold-AR estimates from the weights becomes a worse and worse match to the true activation correlation.

The second reason is that the uncorrelated-input assumption itself is an approximation. The real input covariance is not exactly the identity, and Fold-AR drops the cross-covariance term $\mathrm{Cov}(\tilde{x}_i, \tilde{x}_j)$ in the variance formula. The resulting error in the centroid variance grows with the number of cluster members. Once enough channels are merged per cluster, this error dominates the carefully computed scaling factor.

5 Attempted extension to VGG11-BN

In the proposal, I committed to running the same experiment on a second architecture, VGG11-BN, and to check whether Fold-AR works similarly there. Comparing the result with ResNet18 would tell whether the $\sim 55\%$ breakdown is specific to ResNet's residual structure or a more general property of Fold-AR. In the end, I could not complete this experiment.

Training itself worked. I trained a torchvision-style VGG11-BN (`make_layers + VGG`, `batch_norm=True`, `num_classes=10`) on CIFAR10 for 25 epochs with the same optimizer settings, and a 32×32 forward pass produced valid output. However, the sparsity sweep did not finish: I left the VGG11 sweep running for more than 24 hours on a T4 GPU and saw no observable progress, and at some point the process appeared to halt without an error message.

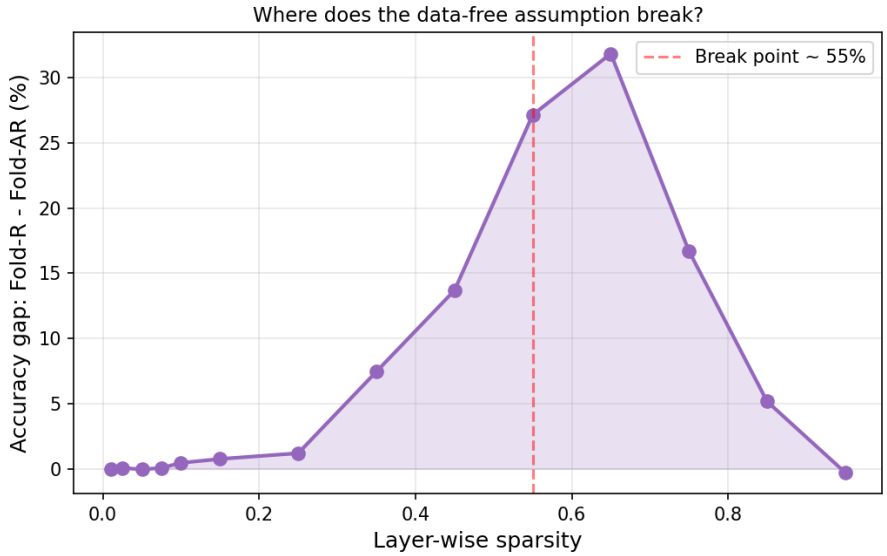

Figure 2: The accuracy gap between Fold-R and Fold-AR as a function of layer-wise sparsity. Below 35% the gap is at most a few percentage points; between 45% and 65% the gap grows by roughly 20 points; at 75% Fold-AR has effectively collapsed while Fold-R still retains 28.84% accuracy. 55% is marked as the qualitative breakdown point.

I could not isolate the cause completely. Possible reasons include:

- **k-means scaling.** My patched code uses sklearn's single-threaded `KMeans`. The widest layers in VGG are 4096-dimensional, while ResNet18's widest conv layers are only 512-dimensional. sklearn's KMeans may scale poorly on the wider VGG layers compared to the ResNet18 case.
- **An unrelated bug in the VGG path.** The ResNet path is explicitly recommended by the universal README, while the VGG path may be less well tested.

Because the VGG11 sweep did not finish, I cannot test whether the $\sim 55\%$ Fold-AR breakdown generalizes beyond ResNet18. This is the most important open question left by this work and is listed again in Section 6.

6 Limitations

Single architecture and dataset. All completed results are on ResNet18/CIFAR10. The VGG11-BN extension proposed in the proposal did not finish, so the $\sim 55\%$ breakdown claim is not verified across architectures.

Different base checkpoint. Because of the structural incompatibility in Section 3, I trained the ResNet18 from scratch instead of using the `PyTorch_CIFAR10` checkpoint that the universal repo's README points to. My base accuracy (94.57%) is about 1.5 points higher than the paper's, so the absolute numbers shift, but the shape of the curves should not change qualitatively.

Fold-DIR not evaluated. The Deep Inversion variant requires a separate pipeline that synthesizes class-conditional images. This report compares only Fold-Naive, Fold-AR, and Fold-R.

Patches may introduce subtle differences. Replacing `HKMeans` with sklearn's `KMeans` should be benign in expectation, since both minimize the same Frobenius objective, but the two implementations differ in initialization and local-move heuristics. I did not run paired ablations.

Single seed per configuration. Each (sparsity, variant) pair is run with a single random seed. The qualitative shape of the curves is stable, but the precise breakdown point ($\sim 55\%$) is not bracketed with seed variance. Small non-monotonic fluctuations visible in the Fold-Naive curve at high sparsity reflect this seed sensitivity but do not affect the main Fold-AR vs. Fold-R comparison.

No proposed fix. This report localizes where Fold-AR breaks but does not propose an improved data-free repair. A natural next step is to estimate a coarse cross-channel covariance from the weights alone—for example from a sample of $\tilde{w}_i \tilde{w}_j^\top$ products—and feed it into the existing scaling formula. I leave this to future work.

7 Conclusion

Model folding is an attractive method: it offers genuinely data-free compression, and the data-free variant Fold-AR is supported by a clean analytical argument. This stress test confirms that, on ResNet18/CIFAR10, Fold-AR is competitive with the data-driven upper bound Fold-R in the low-to-moderate sparsity regime ($\leq 35\%$). It also documents a sharp breakdown of Fold-AR between $\sim 45\%$ and $\sim 75\%$ sparsity, which I attribute to the breakdown of the uncorrelated-input assumption as the cluster size grows.

In addition, the released code turned out to be substantially harder to reproduce than the paper suggests. Four compatibility issues—a dead build dependency (`hartigan-kmeans`), a removed PyTorch exception class, a missing requirement (`thop`), and a structural mismatch with the recommended pretrained checkpoint—had to be resolved before the code could run at all. I document these patches and the retrained checkpoint in the public repository, so that the result is reproducible end-to-end on a current Colab environment.

Acknowledgments

I thank the authors of [1] for releasing their code, which made this stress test possible.

Use of AI assistance. I used a large language model (ChatGPT) for the following: debugging the four compatibility patches in Section 3, polishing the English of this report, and discussing how to structure the analysis. All experimental design choices, model training, sweeps, result interpretation, and conclusions were performed by me.

References

[1] Dong Wang, Haris Šikić, Lothar Thiele, and Olga Saukh. Forget the data and fine-tuning! just fold the network to compress. In *International Conference on Learning Representations (ICLR)*, 2025.

[2] Keller Jordan, Hanie Sedghi, Olga Saukh, Rahim Entezari, and Behnam Neyshabur. REPAIR: Renormalizing permuted activations for interpolation repair. *arXiv preprint arXiv:2211.08403*, 2022.

[3] Hongxu Yin, Pavlo Molchanov, Zhizhong Li, Jose M. Alvarez, Arun Mallya, Derek Hoiem, Niraj K. Jha, and Jan Kautz. Dreaming to distill: Data-free knowledge transfer via deepinversion. In *CVPR*, 2020.

[4] Huy Phan. PyTorch-CIFAR10 pretrained models. `https://github.com/huyvnphan/PyTorch_CIFAR10`, 2021.